PUCE

Pontificia Universidad Católica del Ecuador

PUCE VIRTUAL

Facultad de Ingeniería

Ingeniería en Ciberseguridad

DESARROLLO DE PLATAFORMAS

T1S7 · Unidad 7

Informe técnico de SecureGuard

Depuración, reglas de negocio, ESLint, rendimiento y refactor MVC

500 a 404 · altas concurrentes · Jest 11

Autor

Alejandro David Carriel Zambrano

Carpeta SEMANA 7 · octubre 2026

Quito — Ecuador

1. Objetivo y método

Revisé SecureGuard para reproducir fallos, corregir causas y agregar pruebas contra regresiones. El 401 de un GET sin token y el 400 de un título corto son respuestas correctas, no defectos; quedan como controles negativos. La inspección del modelo encontró dos fallos que podían afectar a usuarios. Las capturas ocultan cualquier token.

2. Defecto 1: identificadores numéricos malformados

Reproducción: sesión iniciada, DevTools > Network y GET `/api/incidentes/1e309` con Authorization. La línea base devolvió 500: `Number(id)` convierte 1e309 en Infinity y el ORM no lo acepta como entero. El valor 1.5 ya devolvía 404; se conserva como caso límite.

Causa: se mezclaba la búsqueda por código y la conversión numérica. Ahora solo se aceptan enteros positivos seguros; el resto se busca como código y, si no existe, responde 404.

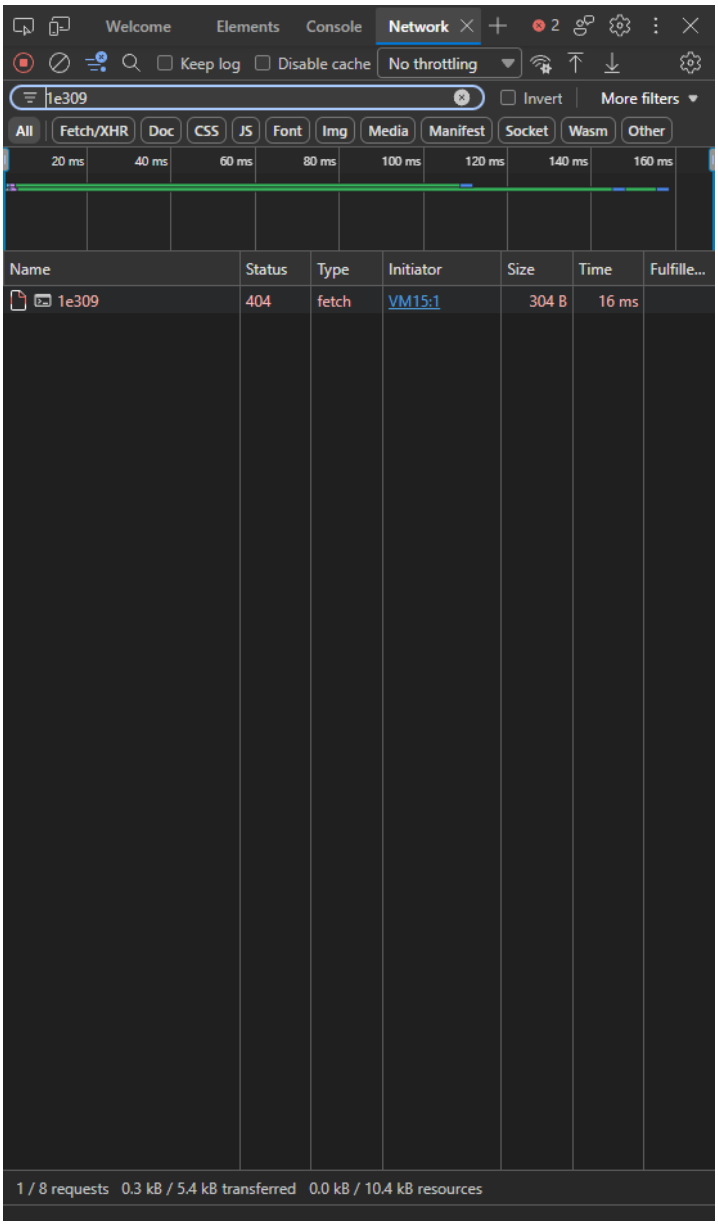


Figura 1. GET /1e309 autenticado: 404, no 500.

La regresión en `tests/incidentes-model.test.js` cubre fraccionarios, enormes, cero, negativos y un entero válido. Tras el parche, 1e309 y 1.5 responden 404; no se envía un ID inválido como número al ORM.

3. Defecto 2: colisión al crear incidentes en paralelo

Reproducción: varias solicitudes POST válidas a la vez. La línea base, con seis, dio una 201 y cinco 500 por colisión de código.

Causa: «leer y luego insertar» no era atómico. Ahora se inserta con código provisional único y se asigna INC-nnn desde el ID autoincremental, dentro de una transacción. Un fallo revierte la inserción.

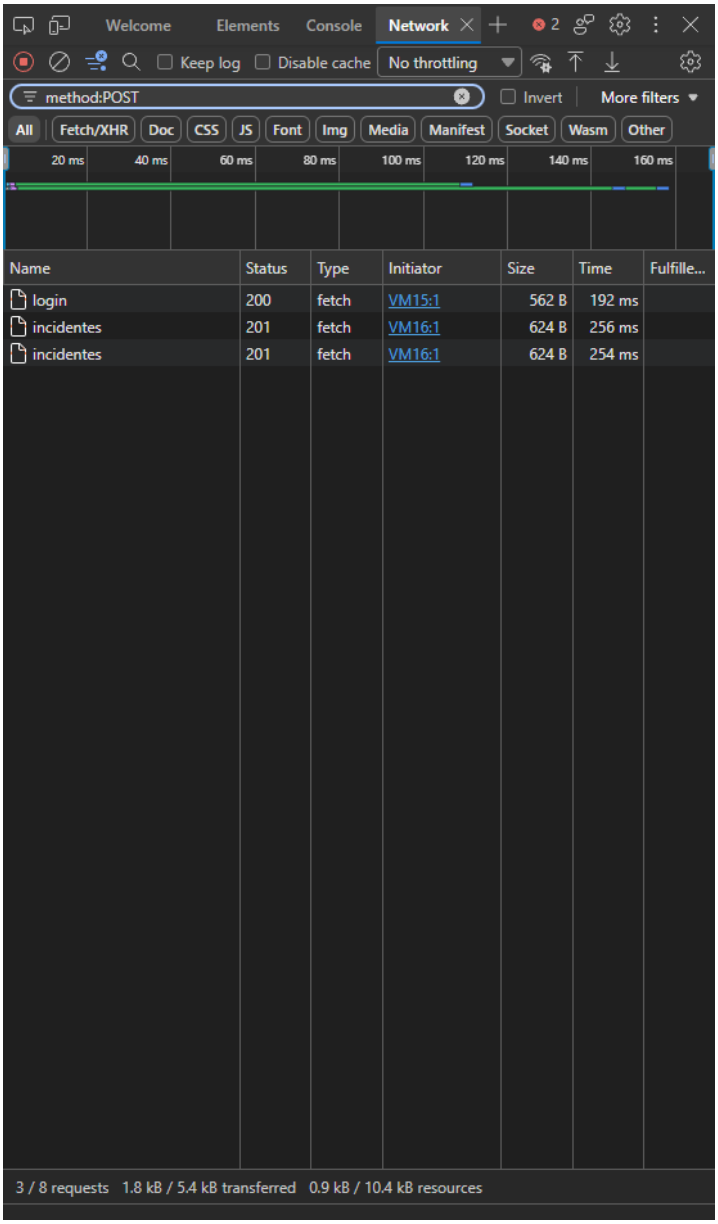


Figura 2. Dos POST simultáneos: ambos 201.

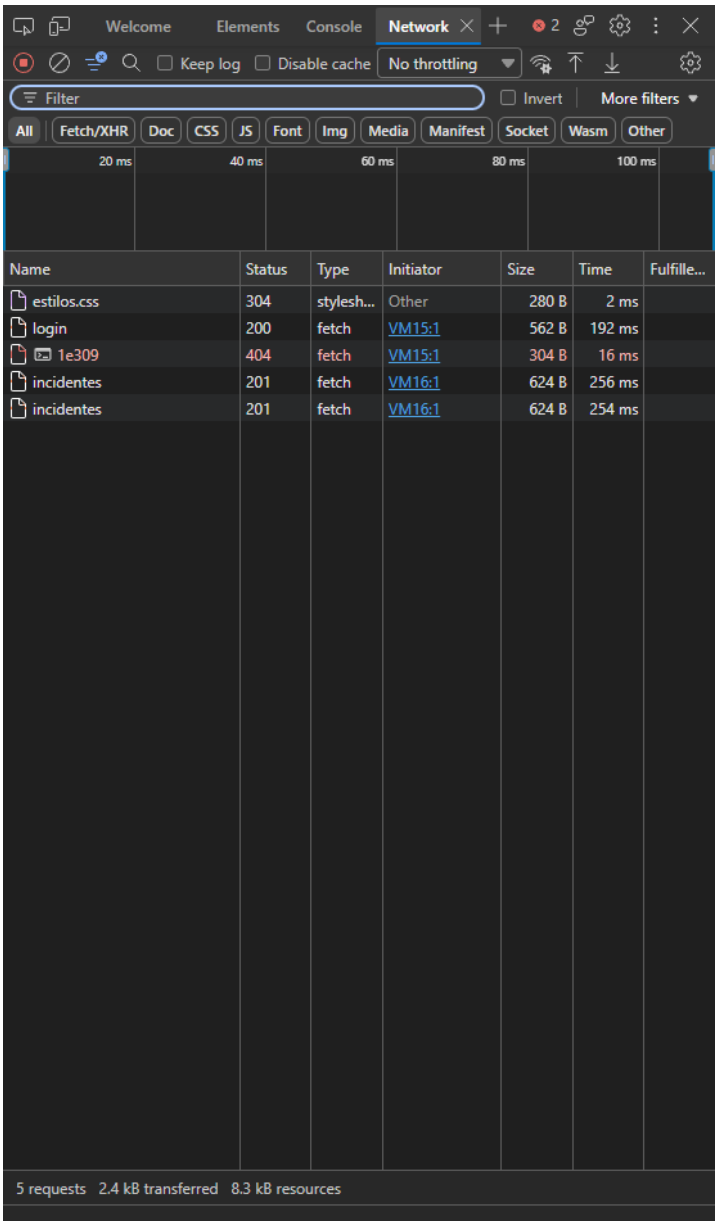


Figura 3. Vista conjunta: 1e309 en 404 y dos altas en 201.

Jest lanza dos altas en paralelo y verifica códigos distintos. Contra MySQL, dos POST simultáneos respondieron 201 con INC-015 e INC-016. Los registros de prueba se eliminaron.

Regresión HTTP contra Express + Prisma/MySQL — 2026-10-15. Línea base (puerto 3000): GET 1e309 → 500 · seis POST → 500, 201, 500, 500, 500, 500. Después (puerto 3001 y luego 3000): GET 1e309 → 404 · dos POST → 201, 201. Códigos INC-015 e INC-016, distintos. Limpieza con DELETE 200. Sin Authorization en evidencia.

4. Controles negativos, no defectos

El listado autenticado muestra 200 y 1223 bytes; sin token, 401 y 38 bytes. No son una comparación de rendimiento. Un POST con título corto responde 400; con título válido, 201. El Bearer se censuró.

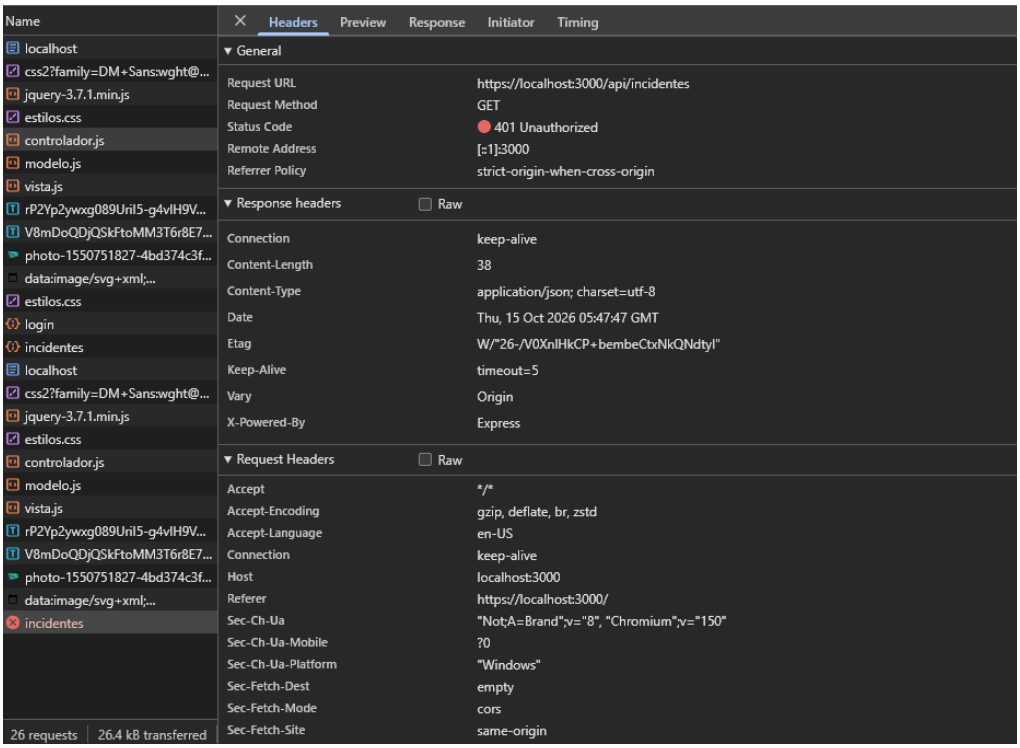


Figura 4. Control: GET sin token, 401 Unauthorized, 38 bytes.

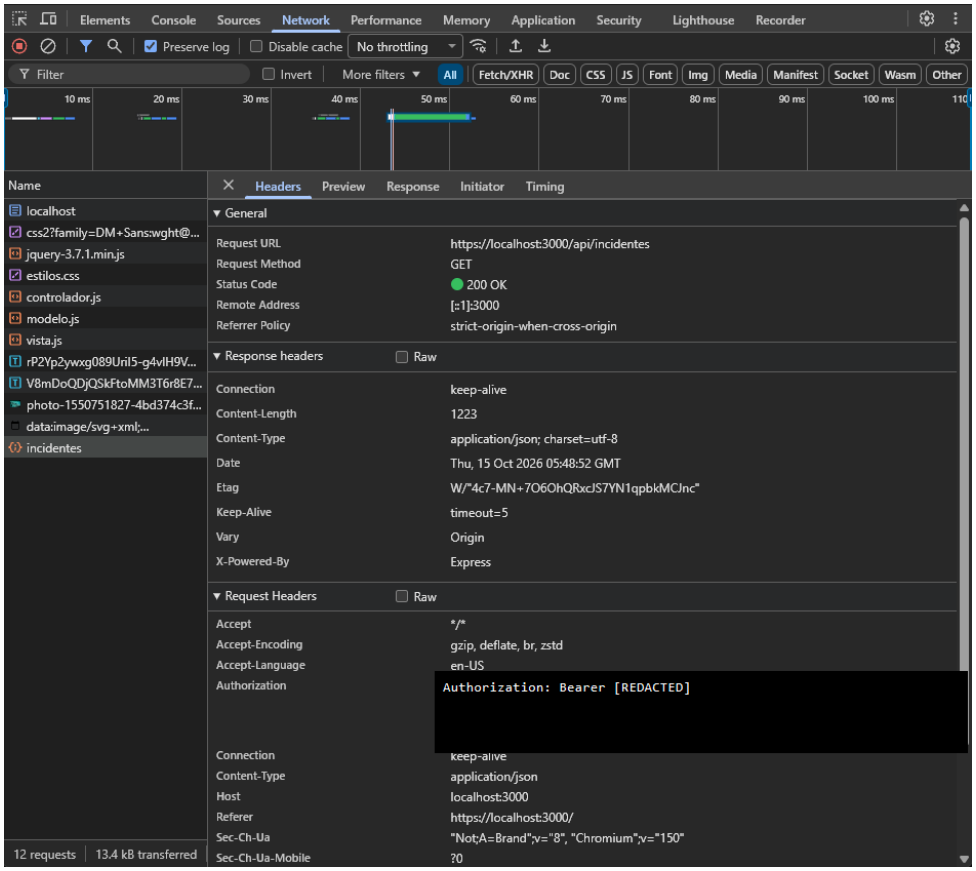


Figura 5. Control: el mismo GET con Bearer censurado, 200.

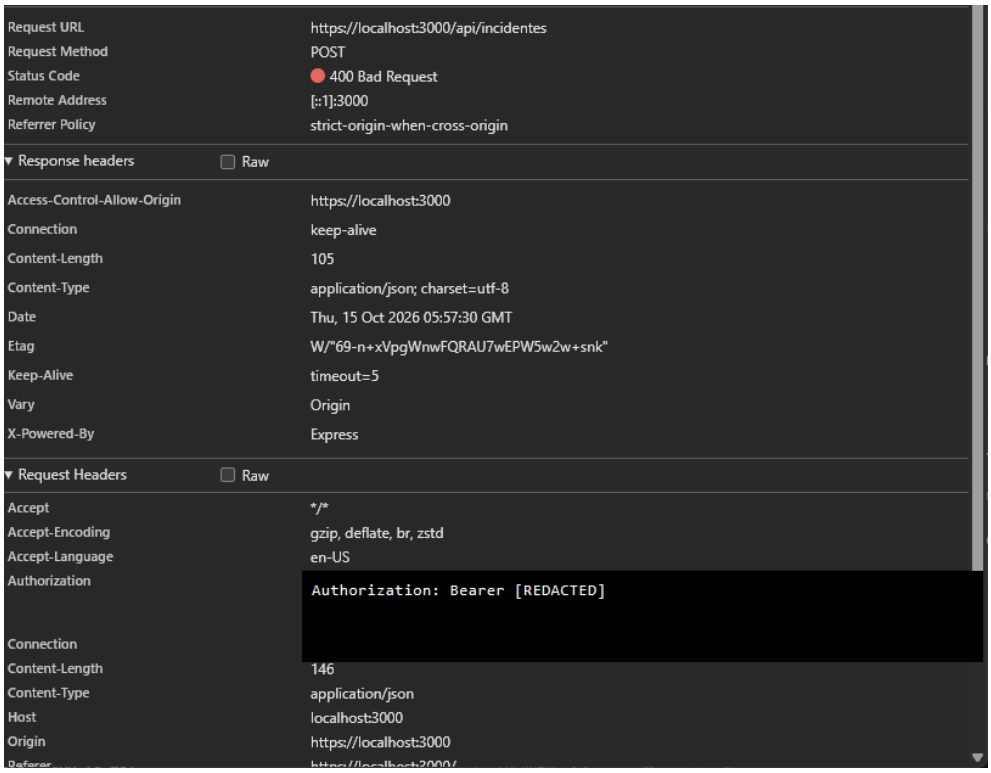


Figura 6. Control: POST con título corto, 400 Bad Request.

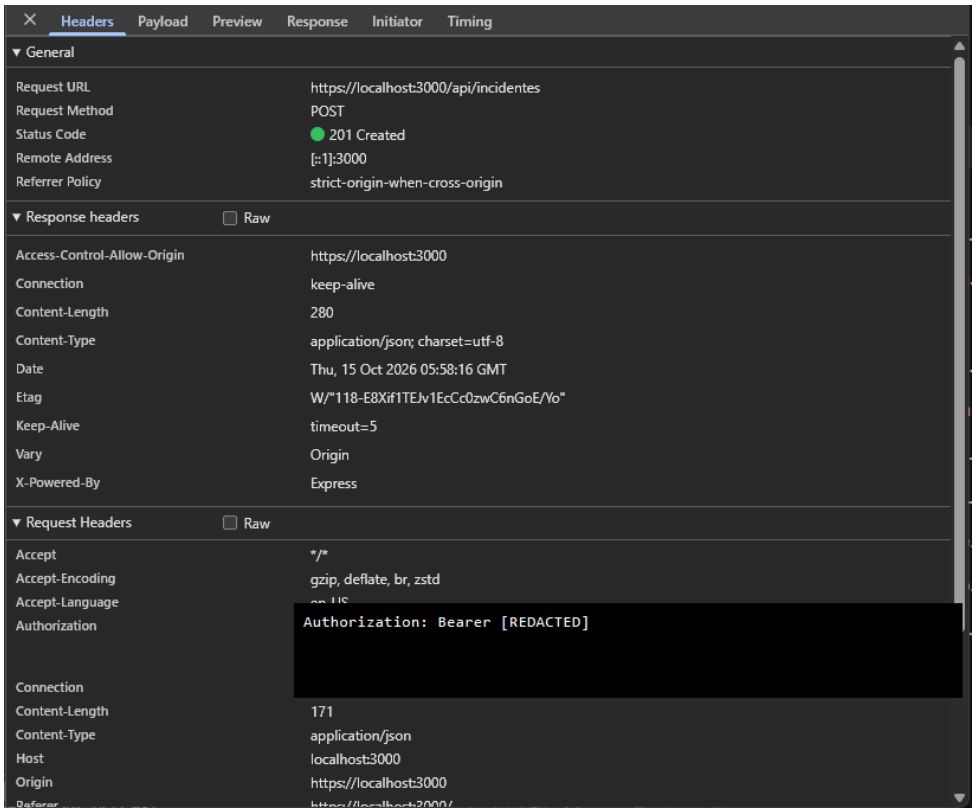


Figura 7. Control: POST con título válido, 201 Created.

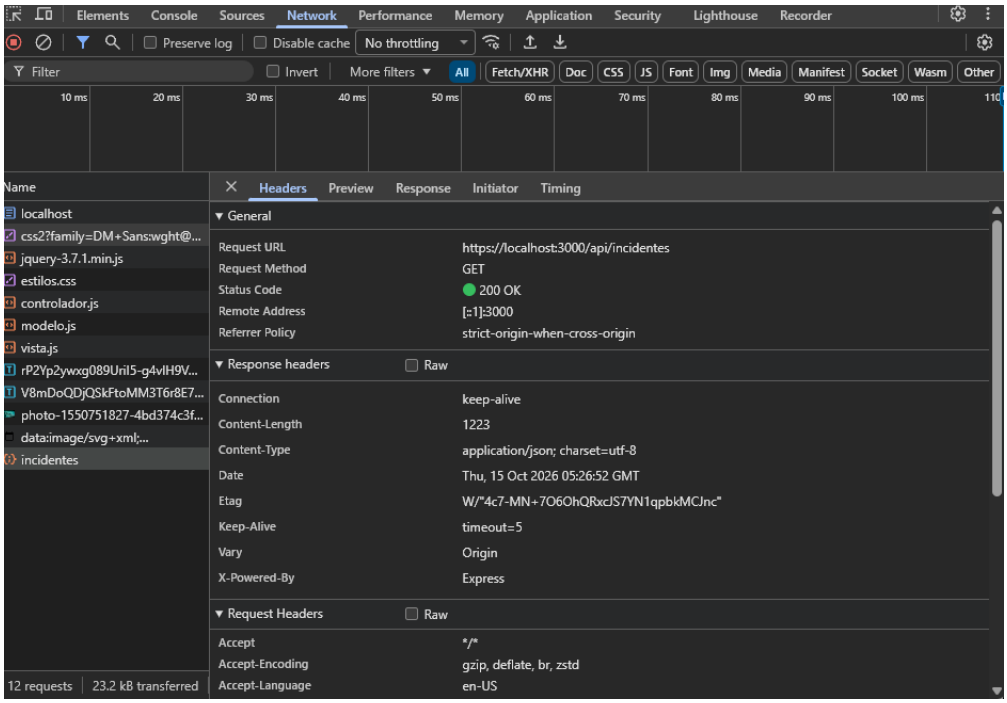


Figura 8. GET autenticado de referencia: 200, JSON, Express.

5. Reglas de negocio y análisis estático

Jest cubre normalización, prioridad, campos ajenos, severidad, fecha, límites del PATCH y el contrato del PUT, más las regresiones de identificadores y altas concurrentes. `npm test --runInBand`: 2 suites, 11 pruebas, exit 0. La línea base de validación tenía 5 pruebas en verde.

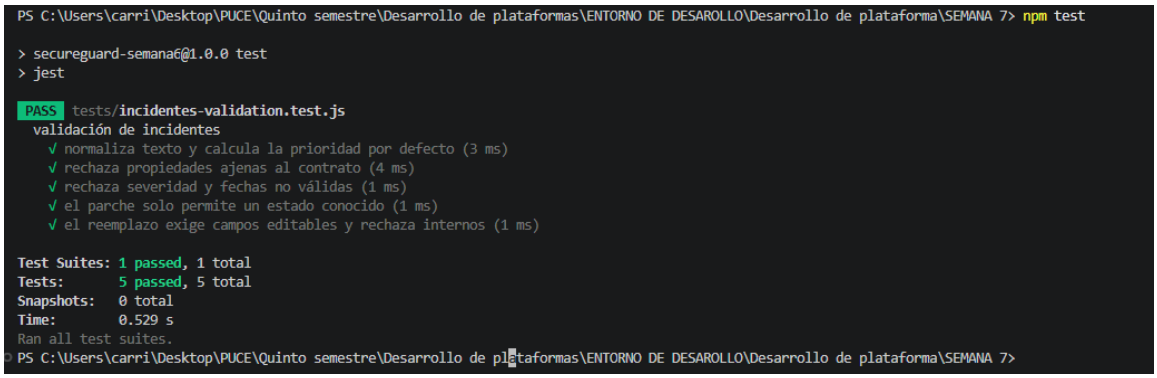


Figura 9. Jest de línea base: 5 pruebas de validación, 0,529 s.

ESLint distingue CommonJS, módulos del navegador y globales de Jest. La captura inicial mostraba 41 diagnósticos, casi todos de parser y entorno; no son 41 defectos de producto. Corregidos, `npm run lint` termina en cero. `.venv/` se excluye.

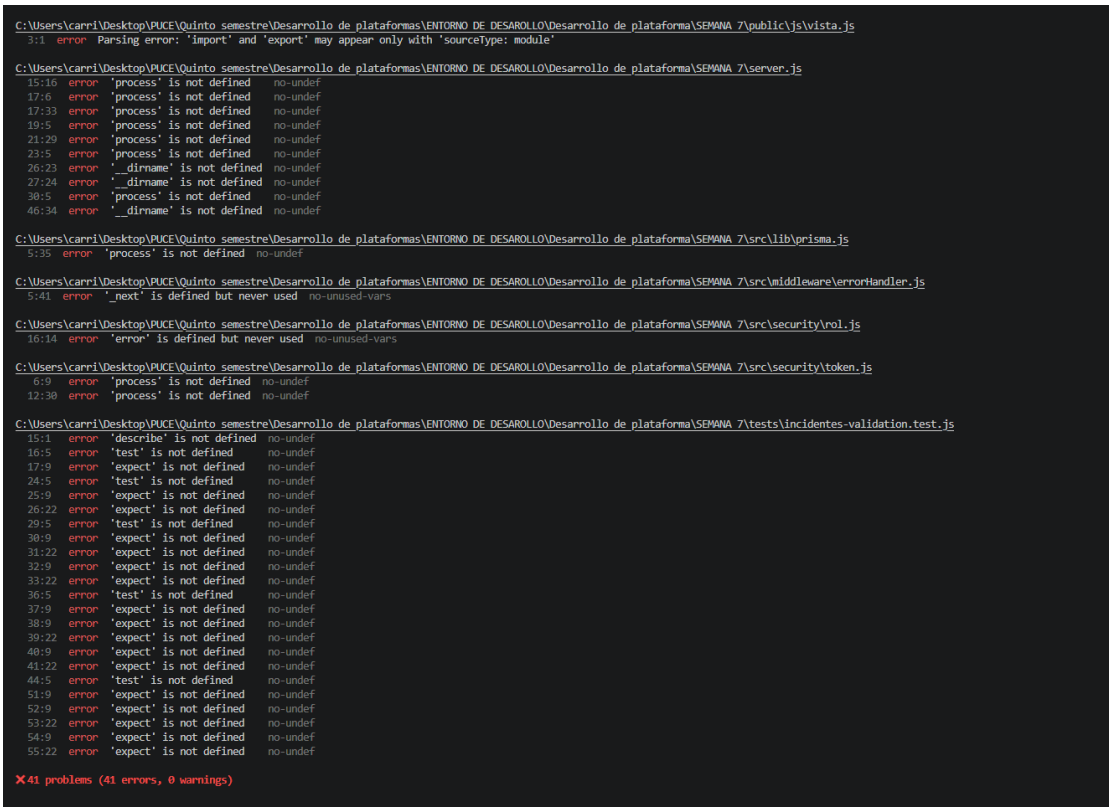
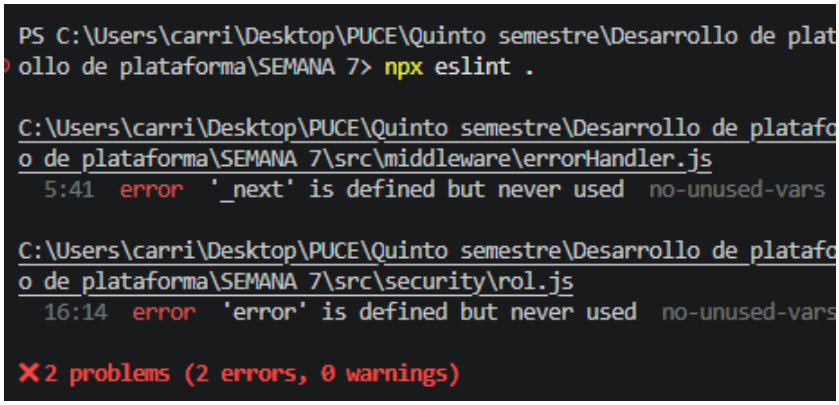


Figura 10. Línea base de ESLint: 41 errores.



Resumen de la línea base: 41 problemas, casi todos de entorno.

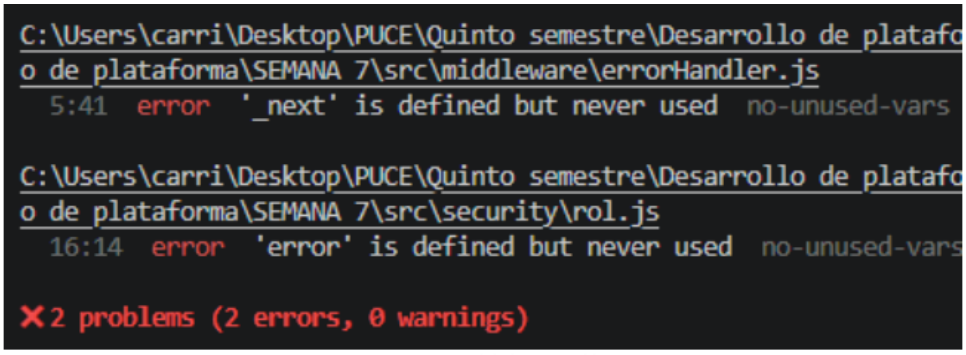


Figura 11. Tras ajustar globals: 2 variables sin uso.

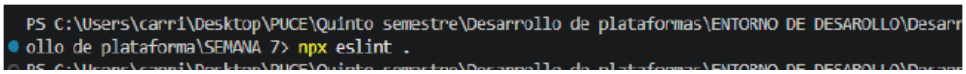


Figura 11. Tras ajustar globals: 2 variables sin uso.

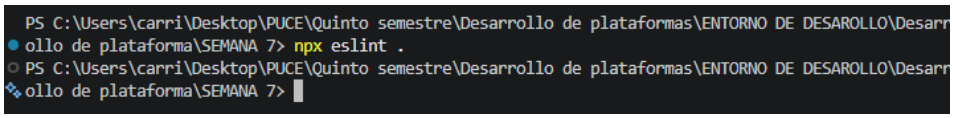


Figura 12. Tras el arreglo: ESLint sin problemas.

6. Mediciones antes y después

El listado se midió con cuatro registros y cinco solicitudes. Antes: 7, 22, 18, 7 y 23 ms; después del select: 7, 22, 15, 8 y 23 ms. La mediana pasa de 18 a 15 ms y la media de 15,4 a 15,0. La muestra es pequeña; no demuestra una mejora significativa.

y 23 ms. La mediana pasa de 18 a 15 ms y la media de 15,4 a 15,0. La muestra es pequeña; no demuestra una mejora significativa.



Figura 13. Listado antes del select: 7, 22, 18, 7 y 23 ms.

Figura 13. Listado antes del select: 7, 22, 18, 7 y 23 ms.

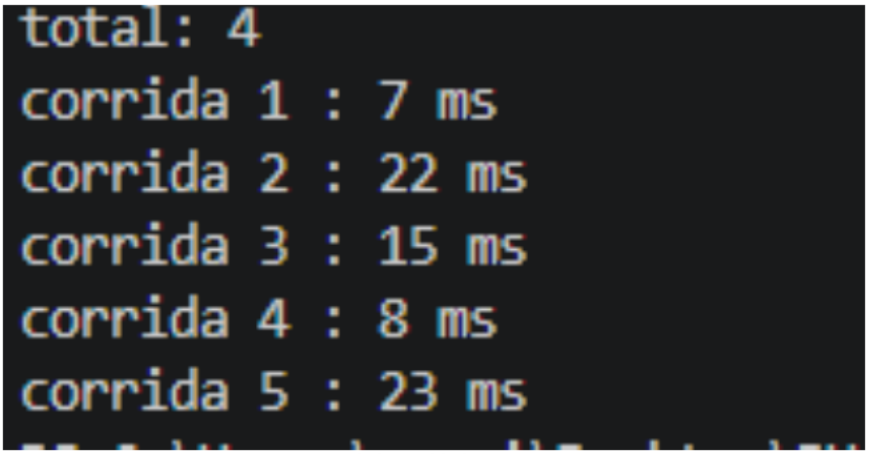


Figura 14. Listado después del select: 7, 22, 15, 8 y 23 ms.

Se redujo el ancho de 1600 a 960. Network reportó 193924 y 107603 bytes: 86321 menos (44,5 %). La captura posterior proviene de caché; es tamaño reportado, no descarga en frío.



Figura 14. Listado después del select: 7, 22, 15, 8 y 23 ms.

Se redujo el ancho de 1600 a 960. Network reportó 193924 y 107603 bytes: 86321 menos (44,5 %). La captura posterior proviene de caché; es tamaño reportado, no descarga en frío.

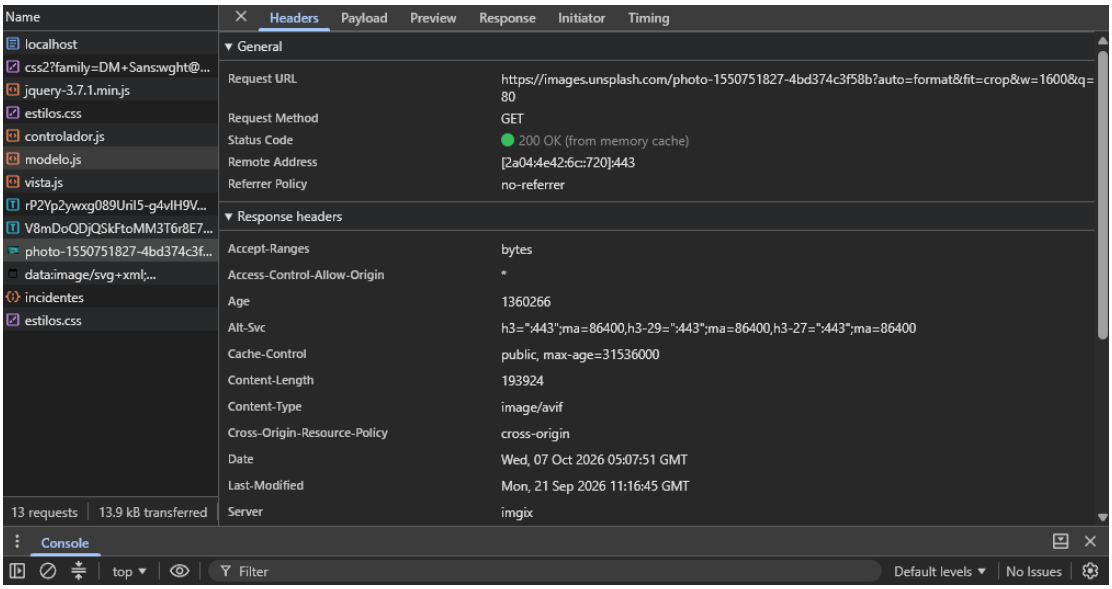


Figura 15. Encabezado antes: w=1600, Content-Length 193924.

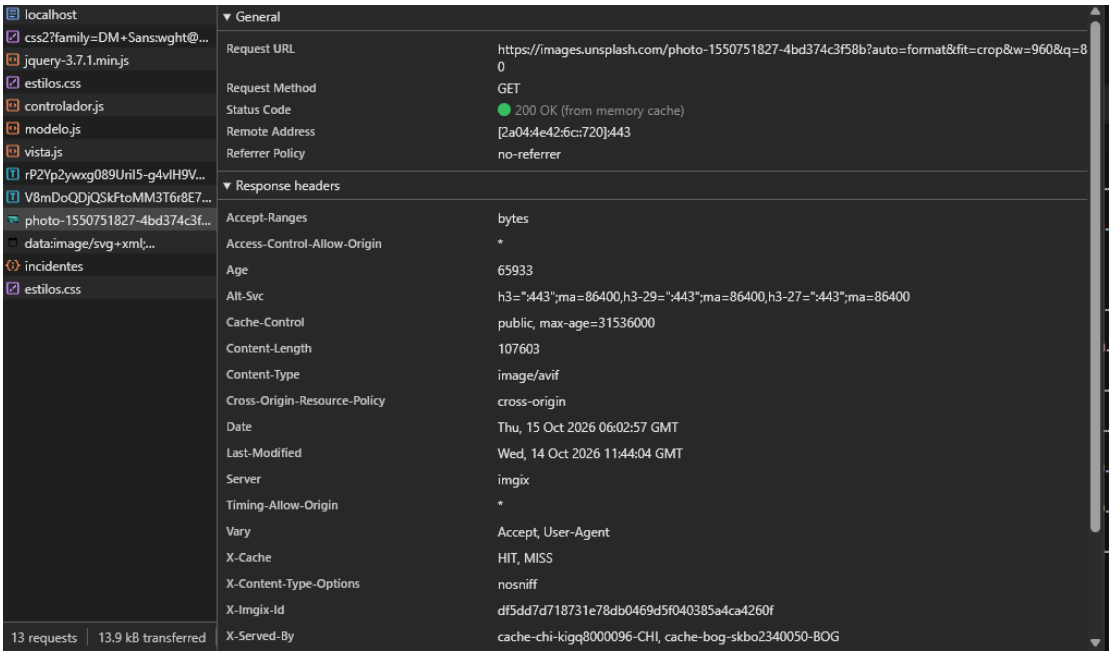


Figura 16. Encabezado después: w=960, Content-Length 107603.

7. Cómo repetir las comprobaciones

Para el primer caso, GET `/api/incidentes/1e309` debe responder 404, no 500. Para el segundo, dos POST válidos en paralelo deben dar dos 201 con códigos distintos; luego se eliminan con rol administrador. La prueba unitaria simula la transacción; el resultado HTTP se verificó también contra MySQL.

8. Estructura MVC y conclusión

La validación está en `src/validation/incidentes.js`, el controlador traduce HTTP y el modelo centraliza persistencia. Ambas correcciones quedan en el modelo. El README distingue errores esperados y defectos.



```
src > controllers > JS incidentes.controller.js > ...
async function listar(req, res, next) {
}

async function obtener(req, res, next) {
    try {
        const incidente = await modelo.obtener(req.params.id);
        if (!incidente) return res.status(404).json({ ok: false, mensaje: "Incidente no encontrado" });
        res.status(200).json({ ok: true, datos: incidente });
    } catch (error) {
        next(error);
    }
}

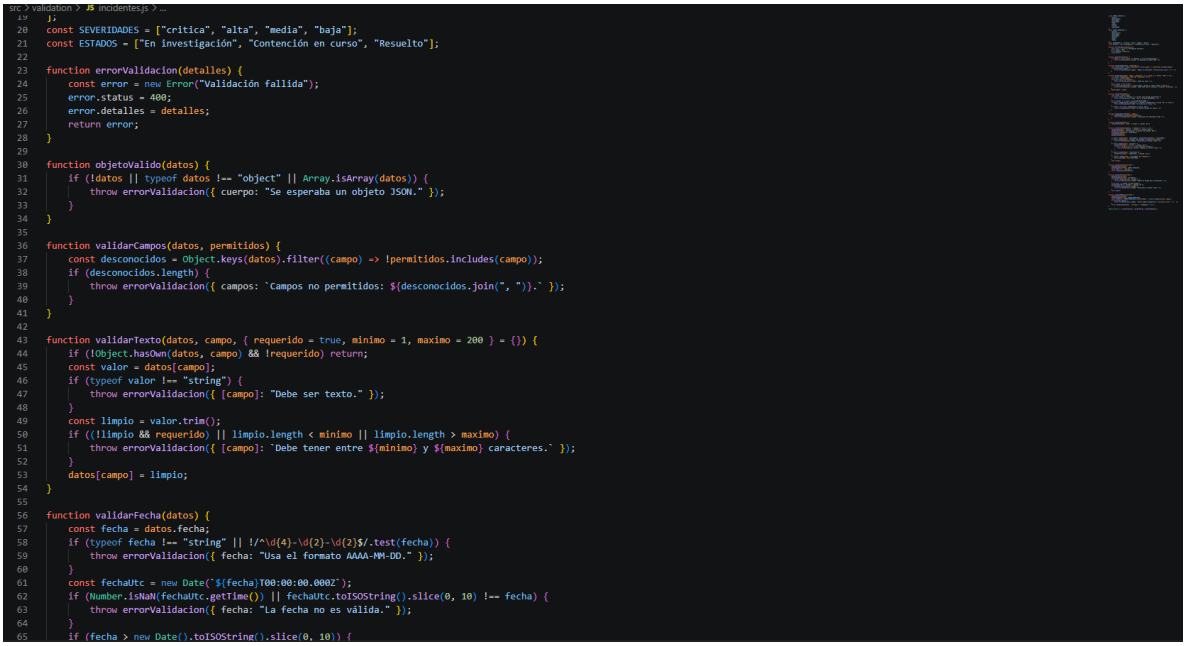
async function crear(req, res, next) {
    try {
        const datos = validarCreacion(req.body);
        res.status(201).json({ ok: true, datos: await modelo.crear(datos) });
    } catch (error) {
        next(error);
    }
}

async function actualizar(req, res, next) {
    try {
        const cambios = req.method === "PATCH"
            ? validarParche(req.body)
            : validarReemplazo(req.body);
        const actualizado = await modelo.actualizar(req.params.id, cambios);
        if (!actualizado) return res.status(404).json({ ok: false, mensaje: "Incidente no encontrado" });
        res.status(200).json({ ok: true, datos: actualizado });
    } catch (error) {
        next(error);
    }
}

async function eliminar(req, res, next) {
    try {
        const borrado = await modelo.eliminar(req.params.id);
        if (!borrado) return res.status(404).json({ ok: false, mensaje: "Incidente no encontrado" });
        res.status(200).json({ ok: true, datos: borrado });
    } catch (error) {
        next(error);
    }
}

module.exports = { listar, obtener, crear, actualizar, eliminar };
```

Figura 17. Controlador: traduce HTTP y llama a la validación.



```
src > validation > JS incidentes.js > ...
};
const SEVERIDADES = ["critica", "alta", "media", "baja"];
const ESTADOS = ["En investigación", "Contención en curso", "Resuelto"];

function errorValidacion(detalles) {
    const error = new Error("Validación fallida");
    error.status = 400;
    error.detalles = detalles;
    return error;
}

function objetoValido(datos) {
    if (!datos || typeof datos !== "object" || Array.isArray(datos)) {
        throw errorValidacion({ cuerpo: "Se esperaba un objeto JSON." });
    }
}

function validarCampos(datos, permitidos) {
    const desconocidos = Object.keys(datos).filter((campo) => !permitidos.includes(campo));
    if (desconocidos.length) {
        throw errorValidacion({ campos: `Campos no permitidos: ${desconocidos.join(", ")}.` });
    }
}

function validarTexto(datos, campo, { requerido = true, minimo = 1, maximo = 200 } = {}) {
    if (!Object.hasOwn(datos, campo) && !requerido) return;
    const valor = datos[campo];
    if (typeof valor !== "string") {
        throw errorValidacion({ [campo]: "Debe ser texto." });
    }
    const limpio = valor.trim();
    if ((!limpio && requerido) || limpio.length < minimo || limpio.length > maximo) {
        throw errorValidacion({ [campo]: `Debe tener entre ${minimo} y ${maximo} caracteres.` });
    }
    datos[campo] = limpio;
}

function validarFecha(datos) {
    const fecha = datos.fecha;
    if (typeof fecha !== "string" || !/^\d{4}-\d{2}-\d{2}$/.test(fecha)) {
        throw errorValidacion({ fecha: "Usa el formato AAAA-MM-DD." });
    }
    const fechaUtc = new Date(`${fecha}T00:00:00.000Z`);
    if (Number.isNaN(fechaUtc.getTime()) || fechaUtc.toISOString().slice(0, 10) !== fecha) {
        throw errorValidacion({ fecha: "La fecha no es válida." });
    }
    if (fecha > new Date().toISOString().slice(0, 10)) {
```

Figura 18. Reglas en src/validation/incidentes.js.

La evidencia está en evidencias/. La ejecución local verificó 11 pruebas, cero errores de ESLint, el paso de 500 a 404 en el ID fuera de rango y dos altas concurrentes correctas.